Savana: Confining Disclosure and Effects in LLM Agents
When the Planner Is Not Trusted Either

Anonymous Author(s)
Submission to USENIX Security '27

Abstract

Tool-using large language model (LLM) agents read attacker-controlled content and act with a user's authority. System-level defenses such as CaMeL separate a privileged planner from untrusted data, but they still trust the planner: whatever the plan writes is authorized. We present Savana, a security kernel that treats every model, the planner included, as an untrusted proposer. The user (the *owner*) signs a task authority with one clause per operation, and the kernel binds every argument of every dispatched tool call to an origin: owner text, an owner-declared constant, or a value the kernel itself derived along an owner-signed edge from an earlier authorized result. Disclosure is mediated by four-dimensional information-flow labels and signed, recipient-bound declassification, and every effect is journaled before its first byte leaves the system.

We implement Savana as five mutually authenticated Rust services. On all 949 injection pairs of AgentDojo v1.2.2, Savana admits no targeted attack and no unauthorized tool request, whereas CaMeL with the same model admits 2. When the planner itself is given hidden attacker objectives, the same model inside CaMeL executes 7 recipient redirections and at least 3 value changes (3 and 2 under CaMeL's policies); inside Savana it executes none, and every attempt is refused before signing. Under AutoDojo's adaptive injections, Savana admits 1 of 389 pairs, an attack on the answer text; when the planner has read the injected data, 8 attacker operations still run under CaMeL's policies and none in Savana. A value-blind planner sees none of the owner's personal data. Mediation costs 59 ms per tool call at the median and 85 ms at p99, and in 461 crash injections no effect repeated or was lost. The cost is utility: Savana completes 40 of 97 benign tasks (41 with deterministic compute steps), against 58 for CaMeL (39 with its policies). We attribute every remaining failure; the largest share follows directly from refusing destinations chosen by data.

1 Introduction

LLM agents combine a model with tools that read data and change external state. This creates a confused-deputy problem: web pages, documents, e-mail, and tool results are attacker-influenced, yet they enter the same model context that drives action selection. Prompt injection exploits this ambiguity to turn data into instructions [38, 21]. Instruction hierarchies, input transformations, and alignment reduce attack success [45, 22, 10, 11], but they leave the model on the authorization path, and adaptive attacks routinely break them [34].

Systems security answers confused deputies by separating decision-making from authority [41, 18, 31]: untrusted components may compute over data, but a reference monitor decides which principals may observe a value and which effects it may authorize [17, 32, 19, 49, 26]. CaMeL [14] brought this idea to agents. A privileged LLM (P-LLM) sees only the user's query and writes a program; a quarantined LLM processes untrusted data; an interpreter tracks capabilities and checks policies before each tool call. Follow-up systems refine the planner, the policies, or the isolation [13, 42, 27, 1, 24].

These designs share one trust assumption: the planner's output is the user's intent. A literal the P-LLM writes into its program is treated as trusted, because the P-LLM saw only the user's query. That assumption fails when the planner itself is compromised—by a malicious or backdoored model provider, by a poisoned fine-tune, or by an injection that reaches it through conversation history or memory. We measured what this means in practice (§10.3). We gave the same model, used as CaMeL's P-LLM, a hidden objective to redirect the user's sends to an attacker. Over 97 benign AgentDojo tasks, CaMeL executed 7 writes that included the attacker: it added the attacker as a meeting participant, shared a file with the attacker, and copied the attacker on an e-mail. Three of these runs were still scored as successful by the benchmark, because the official task checker does not look for an extra recipient. CaMeL's policies blocked four, but only because the same policies already reject those tasks for an honest planner. They admitted the other three, because the attacker's address was a literal in trusted code.

Savana takes the next step: *no model is on the authorization path, including the one that writes the plan.* Three challenges follow.

C1: Authority without a trusted planner.
If the plan is untrusted, something else must say which tools may run, against which resources and destinations, and with which values.

C2: Data-dependent tasks.
Real tasks need values found in data: an IBAN in a bill, the highest-rated hotel, the end time of a meeting. These values must be able to fill a field without being able to choose an effect.

C3: Mediated disclosure and durable effects.
Every model, approver, and executor sees only what policy admits; and the decision to act must survive policy changes and crashes without duplicating an effect whose outcome is unknown.

Savana answers C1 with an owner-signed *task authority*: one clause per operation, fixing the tool, its effect, resource, destination, and bounds. The planner proposes a whole program over a reviewed catalog. A fixed owner review admits or refuses it, but the review is only a pre-filter; the kernel is the reference monitor. For C2, a value may come from data only along an owner-signed *edge*: the kernel itself extracts it from an earlier authorized result, at a signed JSON path, within a signed byte bound, or through a quarantined extractor or a deterministic computation the owner named. Such a value carries untrusted integrity and can never be a destination. For C3, Savana labels every value along four dimensions (integrity, confidentiality, readers, effects), admits audience expansion only through signed and recipient-bound declassification transitions, and journals every effect before its first provider byte.

Results. On AgentDojo v1.2.2 [15] with DeepSeek models, every batch ran on a fresh, disposable host, and every batch passed offline re-verification of its evidence chain.

- **Injections:** Savana admitted no targeted attack and no unauthorized tool request across all 949 injection pairs. CaMeL with the same model admitted 2, both of which push a hotel recommendation through the answer text, which is outside Savana's structural guarantee (§10.2).
- **Compromised planner:** the planner was given four hidden attacker objectives, across 388 episodes plus 194 more with up to three drafts. No attacker effect and no unauthorized request occurred. Every program that encoded an attack was refused before it was signed.
- **Privacy:** the value-blind planner and extractor reduce the personal data that models receive from 27 and 1,093 detected spans to zero.
- **Utility:** Savana completes 40/97 benign tasks, or 41/97 with deterministic compute steps, against 58 for CaMeL and 39 for CaMeL with its policies. With a stronger model, CaMeL rises to 71 and 47; Savana does not improve. Five banking tasks pass even if the agent does nothing; we report scores with and without them.

We attribute every failure. Most follow from two deliberate choices: recipients chosen by data are refused, and a task must complete within a short authorization window.

Contributions.

1. A security-kernel design in which the planner is outside the trusted computing base. An owner-signed task authority and per-value origin enforcement replace trust in the plan. Labels with an untrusted-effect ceiling and control-facet endorsement prevent data or planner choice from minting authority (§5, §6).
2. Owner-signed result edges that let data-dependent tasks run without giving data control. The edges cover JSON paths, typed lists, owner-stated computations, a quarantined extractor, conditional writes, and deterministic compute steps (§6).
3. Signed, recipient-bound declassification and journal-before-effect execution, implemented in five mutually authenticated services (§7–§9).
4. An evaluation of these mechanisms and of CaMeL under the same model, covering the full AgentDojo injection matrix, a compromised planner, model privacy, utility attribution, and cost (§10).

2 Background and Motivation

2.1 Agents, tools, and planners

An agent receives a request from its user, calls tools that read or change external state, and returns an answer. Tool results—e-mails, files, web pages, transaction histories—are written by third parties. An indirect prompt injection places instructions in such a result. The goals are either to *disclose* data to an attacker-chosen sink, or to perform an *effect* the user did not request [21, 15].

Plan-then-execute defenses fix the control flow before any untrusted data is read. In CaMeL [14], the P-LLM translates the query into Python. Tool results are opaque values tagged with capabilities, and a quarantined LLM can turn them into structured values but cannot call tools. Security policies are checked before each tool call. In the reference implementation they are applied by replaying a recorded run. Variants plan over abstract tools (ACE [1]), build tool-dependency graphs (IPIGuard [24]), learn privilege policies (Progent [42]), or label values with information-flow types (FIDES [13]).

2.2 The planner is an authorization input

Table 1 separates useful defenses from the property Savana targets. A plan-then-execute design removes data from the control path, but it keeps the planner on the *authorization* path. Every literal the P-LLM writes is an endorsed value. A recipient it invents is indistinguishable from one the user named. This is a deliberate trust choice. It is not a bug: the P-LLM saw only the query. But it makes the planner's integrity part of the security argument.

Mechanism	What it addresses	Residual authority path
Instruction hierarchy [45]	Conflicting instructions	Model output remains an authorization input.
Input/output detection [22, 29]	Known attack patterns	A missed attack reaches the same action path.
Tool isolation [47]	Tool-code compromise	Authorized arguments may still encode an unsafe effect.
Plan-then-execute [14, 1]	Data-driven control flow	The planner's literals and tool choices are trusted.
Human approval	User consent	Security depends on who constructs the displayed bytes.
Savana	Disclosure and effect mediation	No model output carries a capability; every value has a signed origin.

Table 1: Defense mechanisms are complementary. Savana moves authorization out of every model, including the planner, and mediates disclosure and effects at a reference monitor.

Worked example A. The user asks: *"Please reschedule my Dental check-up to 2024-05-20 at 10:00."* We gave the same planner model a hidden instruction to change amounts and times. Inside CaMeL, the P-LLM wrote `reschedule_calendar_event(event_id, "2024-05-21 11:00")`. The call ran with and without CaMeL's policies, because the literal is trusted code. Inside Savana, the same model drafted the same value. The owner review refused the program before signing, because `2024-05-21 11:00` is not text from the request and is not a value the owner declared. No operation ran (§10.3).

3 Threat Model and Goals

3.1 Adversary

The adversary controls all content the agent reads: web pages, documents, messages, and tool results. It may choose *arbitrary output for every model*: the planner that drafts the program, the quarantined extractor that reads data, and any remote model. This covers prompt injection, a malicious or backdoored model provider, and a planner with a hidden objective. The adversary may gain arbitrary code execution in the agent daemon. It may operate a registered connector and control the bytes that connector returns. It also controls the network: it may observe, delay, replay, reorder, modify, or drop messages. We separately inject crash faults at adversarially chosen instruction and durability boundaries. The adversary knows the design, the review rules, the catalog, and the prompts.

The adversary does not initially possess deployment, policy, service, owner, or WebAuthn signing keys. It cannot break the cryptographic primitives, and it cannot compromise the host kernel, hypervisor, or hardware root of trust. Denial of service is always possible: an agent, model, connector, executor, or network adversary may refuse to make progress.

3.2 Trusted computing base

Savana has a distributed, least-privilege TCB.

- The **owner** is trusted for intent. It states the request and signs the task authority. In our evaluation, a fixed review procedure acts for the owner: it admits or refuses a drafted program and is listed in the TCB (Table 2).
- The **kernel** is trusted to enforce labels, provenance, value origins, policy, and dispatch state.
- The **approval service** is trusted to authenticate the owner and to sign only a confirmation bound to the bytes rendered in its authenticated UI channel.
- The **executor** is trusted to use connector credentials only for a kernel-authorized envelope, and to journal before crossing the external-byte boundary.
- The **signing authorities** for deployment and declassification, the platform's cryptography, and process/IPC isolation are trusted.

No language model is trusted.

Compromising a trusted component invalidates the property it owns. A compromised executor can misuse connector credentials, so it is inside the effect TCB. Mutual authentication prevents that compromise from also minting kernel policy or approval settlements. This is compartmentalized trust, not Byzantine fault tolerance.

Table 2 gives a conservative crate-level accounting at the evaluated commit. It counts complete crates, including inline tests and code not exercised by every property, so it overapproximates the executable slice.

Trusted component group	LOC	Properties
Kernel policy path (Rust)	261,848	S1–S6; policy-core, kerneld, protocol, vault, leak gate, input runtime
Approval service (Rust)	18,235	S1 for owner-endorsed actions
Executor (Rust)	19,525	S1 and S4 at the external-byte boundary
Platform identity (Rust)	13,003	service identity and process isolation
Owner review (Python)	≈950	S6 pre-filter in the evaluated deployment
Conservative union	≈313k	property-dependent distributed TCB

Table 2: Conservative TCB accounting (physical lines, inline tests included). The union is not the TCB of every request; each property relies on the components named in its row.

3.3 Security goals

S1 Effect authorization.
Every side effect corresponds to an exact-action capability prepared under the active signed policy. The capability is justified either by an owner-signed task clause matched in full, or by a matching single-use approval settlement.

S2 Disclosure confinement.
A value crosses a reader boundary only if its label already admits that reader, or an authorized declassification transition admits the exact outgoing bytes.

S3 No authority from untrusted data or choice.
A value with external-untrusted integrity carries no effect beyond READ, including after derivation with a more trusted value. A planner's selection on any of seven control facets is recorded against the planner's output and re-checked before dispatch.

S4 Recoverable attribution.
Every value and authorization decision is linked to bounded provenance and durable evidence. After a crash, the system distinguishes proven no-effect from an indeterminate effect, and never retries the latter automatically.

S5 Fail-closed mediation.
Parse, verification, policy, gate, identity, freshness, and recovery failures do not widen readers or effects.

S6 Origin confinement.
For every dispatched operation and for the final release, the tool, its effect, resource, destination, and bounds are fixed by an owner-signed clause. Every argument value is one of: (a) owner text, i.e. a substring of the request after NFC normalization and whitespace folding; (b) an owner-declared constant; or (c) a value the kernel derived along an owner-signed edge from an already-authorized result, within the signed path, byte bound, list form, and computation.

S2 is a mediation property, not a claim that a heuristic recognizes every semantic secret. For model-facing declassification, the confidentiality claim is parameterized by the deterministic sensitive-content predicate of §7.4. S1, S3, and S6 do not depend on recognizing an injected instruction. A missed instruction may change a model's proposal, but it does not give that proposal a capability.

3.4 Non-goals

Savana does not make model output correct or useful. It does not protect the *wording* of a final answer that the owner asked a model to compose from untrusted data. It does not tolerate compromise of the relevant TCB component, and it does not resist timing or microarchitectural side channels. It does not hide the structure of a plan from the planner that wrote it. It does not control data after a correctly authorized release, and it does not provide availability against a component that refuses service. Within the owner's grant, a compromised planner may still choose among owner-authorized values: for example, it may pick a different record returned by the same authorized read. We measure this residue (§10.3) and do not claim to remove it.

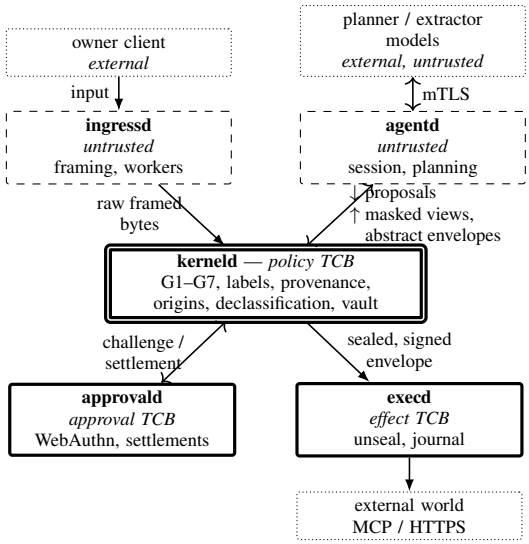


Figure 1: Process decomposition, readable without color. Dashed boxes are untrusted, dotted boxes are external, the double box owns policy, and thick boxes are narrow TCB components for approval and effects. Each edge is a distinct mutually authenticated Suite-1 channel.

4 Overview

4.1 Process decomposition

Savana is a multi-daemon system (Figure 1). Isolation is a property of process boundaries, not module boundaries: a compromised role cannot assume another role's authority, because it does not hold that role's keys or socket credentials.

- `savana-agentd` owns agent sessions and planner orchestration. It is untrusted by design.
- `savana-ingressd` is the entry point for all owner input. It frames the bytes and runs parsers in disposable, sandboxed one-job workers.
- `savana-kerneld` owns labels, provenance, value origins, declassification, the policy gates, and the vault.
- `savana-approvald` owns WebAuthn credentials and signed approval settlements.
- `savana-execd` owns connector credentials, effect dispatch, and the durable execution journal.

A trusted control shell can create tasks and read closed public status, but receives no content and holds no data-plane key.

Eight inter-process edges exist. Each has a distinct client key, server key, peer rule, endpoint role, and operation enumeration. There is no control-shell-to-kernel, control-shell-to-executor, agent-to-executor, or ingress-to-executor data-plane edge. Every edge uses *Suite-1*:

- a fixed 16-byte handshake prefix and an ephemeral X25519 exchange;
- an Ed25519-signed handshake transcript that binds both peers' process boot identifiers and the observed OS peer identity;
- HKDF-SHA-256 traffic keys with mutual HMAC confirmation;
- ChaCha20-Poly1305 records, with exactly one encrypted request and one encrypted response per connection.

4.2 Two enforcement paths

Interactive actions. An agent proposes a tool class and argument handles. The kernel resolves the class against the active signed registry, matches the action against the owner's task authority, and records the planner's choice on each of seven control facets (§5.6). G5 validators then decide whether the action is permitted or needs approval. G6 settles an approval, and G7 declassifies the execution envelope, re-checks the endorsements against the current state, and prepares the effect durably.

Owner-signed task programs. This is the path we evaluate. The planner drafts a whole program before any data is read (§6).

1. The owner signs a task authority with one clause per operation, plus a final release.
2. Signed operator commands compile the program against those clauses and prepare it. Preparation checks the origin of every input value *before any operation runs*.
3. The kernel then drives the operations itself, one at a time, through the same G4–G7 path. Values taken from earlier results are minted only by the kernel.

In both paths the agent and the planner hold no capability-bearing credential, no stored plaintext, and no tool transport.

The implementation assigns these checks to seven gates:

- G1, input normalization;
- G2, local semantic extraction;
- G3, labels and declassification;
- G4, matching the action against the task authority and binding it;
- G5, internal validators;
- G6, human approval;
- G7, durable execution and recovery.

Each gate has a closed request vocabulary. No generic "invoke handler" operation crosses a service boundary.

4.3 Security argument

We state the argument over a durable event trace τ. The relevant events are:

- value construction and derivation;
- a *reveal* of exact bytes to a reader;
- preparation of an exact-action capability;
- the executor's first provider byte;
- recovery classification.

P1 Label closure.
Every ordinary derivation is non-widening in readers and effects, and every externally untrusted value satisfies the READ-only ceiling.

P2 Mediated disclosure.
(a) *Reveals.* If τ reveals bytes b of value v to a system reader r (a component or an exact model, executor, or sink identity), then either $r \in readers(v)$ before the reveal, or τ contains an earlier declassification node for the same bytes, rule set, purpose, and recipient. (b) *Releases through effects.* If action a carries bytes b to its external destination d, then d is the owner-endorsed Destination facet of a, the bytes reached the executor through BuildExecutionEnvelope for a's exact executor, and, if any argument of a is ExternalUntrusted or d was chosen by such a value, τ contains a verified ActionApproval over a's exact request bytes (§5.5).

P3 No data- or selection-derived authority.
An externally untrusted value may occur as an argument, but it cannot by itself justify a control facet or authorize an effect. For every dispatched action, τ contains (i) the seven control selections, recorded against the planner's committed output, and (ii) endorsements that were re-checked against the current state immediately before durable preparation.

P4 Authorized emission.
If the executor emits the first provider byte for action a, τ contains an earlier capability and durable preparation for the same action digest, executor, manifest, fence epoch, and execution nonce.

P5 Crash containment.
For one execution nonce, the executor opens at most one provider attempt. After a crash past the first provider byte, the outcome is effect-started or indeterminate, and Savana never retries it automatically.

P6 Origin confinement.
For every operation of an owner-signed program, each argument the executor receives equals one of three things: an owner-text item, an owner-declared constant, or the kernel's own derivation along the signed edge for that argument. In the third case, the derivation is taken over the retained result of an earlier operation of the same authority.

P1 follows by induction over the closed value constructors. P2(a) reduces every reader-widening route to the pri-

vate declassification constructor; P2(b) reduces to the Destination endorsement and to G5's fail-closed floor (§5.6). P3 reduces each facet to its selection record and the endorsement re-check at G7. P4 and P5 reduce the external-byte boundary to the kernel and executor journals (§8.4). P6 reduces to two checks.

- **Preparation** runs the owner-text origin check before anything executes, and the result-edge rule: the owner's exact values, plus the declared edge rules, must reconstruct an alternative the owner signed.
- **Dispatch** re-matches each request against its clause, where a result-derived value can be produced only by the kernel's own result derivation.

These are implementation arguments, not a mechanized proof (§11).

4.4 P3 as robustness against an untrusted decider

P2 and P3 are instances of a property studied for two decades in language-based security. *Robust declassification* [48] requires that an attacker who controls low-integrity inputs cannot influence *what* information is released. *Qualified robustness* [33] relaxes this to allow untrusted data to affect a release only through an explicit endorsement, and *nonmalleable IFC* [9] adds the dual, *transparent endorsement*: a principal may only endorse information it can read. Askarov and Myers characterize both in terms of attacker control [5]; Sabelfeld and Sands classify release policies by *who*, *what*, *where*, and *when* [39].

We use the same vocabulary. The *decision* of an action consists of its seven control facets and, for any action that releases data, its recipient. P3 says that no low-integrity input—and here every model output is low-integrity—influences that decision except through owner evidence:

- ExplicitAlternative and CompleteContractSingleton are decisions the owner made *before* any untrusted byte was read: the candidate domain is the signed clause and nothing else (§5.7). They correspond to robust declassification, where the release point and its policy are fixed in trusted code.
- ActionApproval is an endorsement of exact bytes on an authenticated display. It corresponds to qualified robustness, and the display transition (BuildApprovalDisplay) makes it *transparent*: the owner endorses only bytes it has been shown.

What is new. Classical robustness assumes that the program is trusted and the attacker controls only some of its inputs; the release points are written by the programmer. In an agent, the "program" is a plan written by a model, and we assume that model is the attacker. Three consequences follow, and they are the parts of the design that the classical results do not provide.

1. **The code is an input.** Robustness must hold when the attacker also chooses *which* release points exist. Savana therefore never lets a release point originate in model output. The owner signs the program's control skeleton (the task authority), and every planner choice is recorded as a selection fixed at ExternalUntrusted and READ, which no derivation can consume.
2. **Robustness is enforced dynamically, per action, and across restarts.** The kernel re-checks endorsements against the current authorization revision, pre-state, and candidate-domain digest immediately before durable preparation. A type system checks robustness once, at compile time; we must check it at the moment of effect, against durable state that an adversary can try to race or replay.
3. **Effects are releases.** An effect discloses its arguments to its destination. We treat the destination of every effect as a reader (§5.5), so the same robustness condition governs both *what* leaves (P2) and *what happens* (P3).

5 Information-Flow Labels

5.1 Dimensions

Every value carries a label $\langle int, conf, readers, effects \rangle$ (Table 3).

- **Integrity** is a three-point total order in which *higher* means *less* trusted, with join $= \max$.
- **Confidentiality** is a four-point lattice that is deliberately non-linear. PlannerAbstract and AgentMasked are incomparable and join to VaultBound. A value derived from both something the external planner may see and something the agent may see is therefore readable by *neither*. The safe outcome falls out of the algebra rather than from a special case.
- **Readers** and **effects** are closed bit sets over seven principals and seven effects, respectively.

Dim.	Structure	Values
int	total order, $\sqcup = \max$	KernelTrusted $<$ UserAuthorized $<$ ExternalUntrusted
$conf$	lattice	Public $\sqsubseteq$ {PlannerAbstract, AgentMasked} $\sqsubseteq$ VaultBound
$readers$	bit set (7)	KERNEL, AGENT, INGRESS, APPROVAL_DISPLAY, EXECUTOR, EXTERNAL_PLANNER, EXTERNAL_SINK
$effects$	bit set (7)	READ, CREATE, UPDATE, DELETE, SEND, EXECUTE, FINAL_RELEASE

Table 3: The four label dimensions.

5.2 Propagation narrows

For parents $p_1, \ldots, p_n$ and the session's signed effect allowance A, ordinary derivation computes

$$int' = \bigsqcup_i int(p_i), \qquad conf' = \bigsqcup_i conf(p_i),$$
$$readers' = \bigcap_i readers(p_i), \quad effects' = A \cap \bigcap_i effects(p_i).$$

The kernel then normalizes $effects'$ according to int', as described below. An ordinary derived value is therefore readable only by readers of every parent, and carries only effects of every parent. Declassification is the only operation that may replace this intersection with a larger audience.

5.3 The untrusted-effect ceiling

Consider an agent that fetches a web page containing an attacker-supplied address, combines it with a user-authorized document handle, and proposes to send. Suppose values were initialized directly from the session allowance. Then the fetched address would carry SEND, and later policy checks would see an apparently authorized recipient. We call this *effect laundering*: attacker-controlled data acquires authority because effect permission is tracked independently of integrity.

Savana enforces

$$int(v) = \mathsf{ExternalUntrusted} \;\Rightarrow\; effects(v) \subseteq \{\textsc{read}\}$$

at every label-construction boundary, including after derivation joins (Listing 1). Source initialization is closed:

- signed policy constants begin KernelTrusted;
- approved owner ingress begins UserAuthorized (approval authorizes *use*; it does not assert truth);
- planner output, tool results, values derived from results, and values recovered after a crash are *always* ExternalUntrusted.

The fact that the kernel performed a computation is never evidence for an integrity upgrade. Given valid parents, effect intersection already preserves the ceiling. The post-join normalization is applied anyway, so that the invariant does not depend on every present and future constructor initializing effects correctly.

5.4 Source labels and the planner's output

Table 4 gives the label every source kind receives. Two rules matter most for the planner.

Planner output. A value the planner writes—a literal, a plan, a program—is labeled

$$\ell_P = \big(\mathsf{ExternalUntrusted},\ \mathsf{PlannerAbstract} \sqcup \bigsqcup_i conf(p_i),$$
$$\{\textsc{kernel}\},\ effects \subseteq \{\textsc{read}\}\big),$$

where p_i are the values in the planner's envelope. The rule has four consequences.

Listing 1: Label derivation (abridged from `labels.rs`). The ceiling is re-applied after the join.

```
pub const UNTRUSTED_EFFECT_CEILING_V2: EffectSetV2 =
     EffectSetV2::READ;

const fn confined_effects(i: IntegrityV2, e:
     EffectSetV2) -> EffectSetV2 {
    match i {
        ExternalUntrusted => e.intersection(
     UNTRUSTED_EFFECT_CEILING_V2),
        UserAuthorized | KernelTrusted => e,
    }
}

fn derive_normal(parents: &[Self], allowed: EffectSetV2
     ) -> Result<Self, G3Error> {
    let (first, rest) = parents.split_first().ok_or(
     G3Error::EmptyParents)?;
    let mut d = *first;
    for p in rest {
        d.integrity       = d.integrity.join(p.
     integrity);            // max
        d.confidentiality = d.confidentiality.join(p.
     confidentiality); // join
        d.readers         = d.readers.intersection(p.
     readers);
        d.effects         = d.effects.intersection(p.
     effects);
    }
    d.effects = d.effects.intersection(allowed);
    d.effects = confined_effects(d.integrity, d.effects
     ); // after the join
    Ok(d)
}
```

1. **Readers.** Planner output is readable by the kernel alone, whatever the planner saw. It cannot be handed to any component, model, executor, or sink except through a declassification transition that names the exact recipient (§7). The planner's readers do not "inherit" from its inputs: they are the minimum.
2. **Confidentiality.** The join with PlannerAbstract records that the bytes were produced by a remote model. A planner value combined with an agent-visible value joins to VaultBound, so it can be shown to neither model (§5).
3. **Integrity.** Planner output is never more trusted than a tool result. Hence a planner-written value that reaches an argument of a state-changing call, or the destination of any call, escalates at G5 (§5.6).
4. **Effects.** By the ceiling, planner output authorizes nothing. Its only path to an effect is as the *selection* recorded against it, which needs owner evidence.

In the evaluated task-program path the planner cannot write an argument at all (S6): its output labels the program, and every argument is owner text, an owner constant, or a kernel derivation. The rule above governs the interactive path and is what makes S6 a defense in depth rather than the only line.

5.5 Effects are releases

Readers name system components, not people: there is no reader bit for Bob. An effect, however, discloses its argu-

Source	int	$conf$	$readers$	$effects$
gated ingress	UA	VB	K	A
policy constant	KT	decl.	decl. $\cap$ pol.	A
planner output	EU	PA $\sqcup \bigsqcup_i conf(p_i)$	K	$\leq$READ
tool result	EU	VB	K	$\leq$READ
extraction, derived	$\bigsqcup$	$\bigsqcup$	$\bigcap$	ceiling
declassification	parents	target	target	parents
recovered execution	EU	VB	K	$\leq$READ

Table 4: Labels at each source (`provenance.rs`). UA/KT/EU: UserAuthorized, KernelTrusted, ExternalUntrusted; VB/PA: VaultBound, PlannerAbstract; K: the kernel only; A: the session's signed effect allowance; ceiling: the untrusted-effect ceiling.

ments to its destination. We therefore treat every effect as a release to its Destination facet, and mediate it with the same three questions a declassification answers: *who* receives (the endorsed destination), *what* bytes (the exact business request), and *who decided* (owner evidence).

Consider a planner that tries to send the content of a secret S to Bob.

- If the planner writes S's bytes as the body, they are planner output, so the call escalates: the owner sees the exact body and recipient before anything is sent. The planner sees S only if a declassification gave it S; under value-blind planning it sees a placeholder (§6.5).
- If the body is bound along a signed edge to a result containing S, the body is ExternalUntrusted by P1, and the call escalates in the same way.
- If Bob is not the owner's recipient, the Destination facet has no owner evidence, so G4 refuses the action before G5 runs. If Bob was chosen by data, the address floor escalates even on a read.

The confidentiality label is *not* compared with the destination: an owner who signs "send my report to Bob" has declassified the report to Bob. P2(b) is therefore an owner-mediation property, not a label-flow property, and we state it as such.

5.6 Control facets and endorsement

Effect labels govern what may *authorize* an action, not which bytes may be carried as an argument. Each action is described along seven closed *control facets*: Tool, EffectKind, Scope, Magnitude, Argument, Destination, and Trigger (`ControlFacetV2`). A signed tool descriptor assigns each business field a role: resource, destination, magnitude, payload, or parameter. A caller can neither omit a facet nor reclassify a field.

Selections. When G4 matches an action against the owner's task authority, it records one `ControlSelectionV2` per facet. Each record binds the selected content digest to the digest of the planner's committed output (the *proposer parent*). Selections are fixed at ExternalUntrusted and READ. They are deliberately *not* provenance nodes that later derivations could consume, so the planner's choice cannot disappear as control flow or be laundered into a value.

Endorsement. An action may proceed only with seven `ControlEndorsementV2` records, one per facet. These are produced by `checked_control_endorsements_v2` from exactly one form of evidence:

- ExplicitAlternative: the action is an alternative the owner wrote out in full;
- CompleteContractSingleton: the owner's contract admits exactly one complete alternative with exactly one allowed positive magnitude;
- ActionApproval: a verified approval for this action and its expected context.

The function first re-checks the whole-action match against the current state, including the authorization revision, pre-state, and candidate-domain digest. It then recomputes the expected selections. Any mismatch fails, and G7 calls it again immediately before durable preparation, mapping a change to `StateConflict`. Endorsements and ordinary values are separate types, and a `compile_fail` test pins that separation.

G5's intent-flow fact closes the remaining path. It is a kernel *floor*, met with every other G5 contributor, and does not depend on a descriptor declaring the validator. A call escalates to approval when (i) it has a state-changing effect and binds *any* ExternalUntrusted argument, or (ii) whatever its effect, an ExternalUntrusted value fills a field the signed profile does not name as a non-destination field. Such a call cannot be permitted silently, even when every facet matches.

5.7 Candidate completeness

An approval-free endorsement is sound only if the owner's evidence determines the action's controls *before* an adversary can influence them. We make the condition explicit.

Owner-closed domains. The candidate domain of a clause is the list of its alternatives in the signed contract and nothing else (`candidate_domain`). Its digest binds the whole canonical contract, its revision, and the clause; no API accepts a subset, a search result, a runtime value, or a caller's assertion of completeness, and G7 re-checks the digest before durable preparation. The domain is therefore fixed when the owner signs, before any tool result exists, and a tool result cannot remove or add a candidate.

Value-closed versus rule-closed alternatives. An alternative commits to resource, destination, and parameter digests.

For an owner literal the digest is over the value; the alternative is *value-closed* and denotes exactly one request. For a result-derived control the digest is over the signed *rule*—source clause, path, bound, computation—under a separate domain, so a literal can never collide with it. Such an alternative is *rule-closed*: it denotes whatever value an adversary places at the signed path. Counting alternatives therefore says nothing about how many concrete requests a rule-closed alternative admits.

The completeness bound. Let $V(f)$ be the set of concrete values the owner's evidence admits for facet f. Approval-free evidence (ExplicitAlternative or CompleteContractSingleton) requires $|V(f)| = 1$ for every facet that can change external state or choose a recipient, and $V(f)$ must be a function of the signed contract alone. Value-closed alternatives satisfy this by construction. A rule-closed control is filled by a kernel derivation from a tool result, which is ExternalUntrusted (P1), so the G5 floor sends it to ActionApproval whenever it steers a state-changing call or fills a destination. The only rule-closed controls that remain approval-free are non-destination fields of reads: data may choose *which* record a fixed provider returns, which is the residue of §6.6. CompleteContractSingleton is implemented and tested in the policy core, but the evaluated deployment never selects it; every approval-free action in our evaluation uses ExplicitAlternative.

5.8 Provenance

Each value has a provenance record. It names the value's source kind, its parents and root evidence digests, its label, the producing service identity, the durable run, the active state-manifest digest, and an expiry. There are eight source kinds:

- gated ingress, kernel extraction, and policy constant;
- planner output and tool result;
- derivation and kernel declassification;
- recovered execution.

The bounds are compiled in: at most 256 parents and 64 root-evidence entries, plus bounded depth, node count, and encoded size. A hostile lineage therefore cannot exhaust the kernel. The kernel refuses a parent from another run or manifest generation, and a parent whose value digest does not match its record.

A value taken from an earlier result is a derivation with operation `SelectResultJsonEdgeV04` over the ToolResult parent, at a signed path and byte bound. It is minted only by the kernel, and it is ExternalUntrusted by P1.

Listing 2: A program drafted by the untrusted planner for "recommend the highest-rated hotel in Paris (the more expensive on a tie) and e-mail it to my wife at janeLong@google.com" (abridged).

```
1 dojo.travel.hotels        city: {"text":"Paris"}
2 extract  source 1  -> dojo.travel.hotel_reviews.
    hotel_names
3 dojo.travel.hotel_reviews hotel_names: {"from":2}
4 dojo.travel.hotel_prices  hotel_names: {"from":2}
5 dojo.compute.table  input1:{"from":3} input2:{"from
    ":4}
                    order1:"max 1" order2:"max 2"
    output:"key"
6 dojo.travel.hotel_address hotel_name: {"from":5}
7 extract  source 5  context [3,4,6]  -> answer
8 dojo.email.send  recipients: {"text":["
    janeLong@google.com"]}
                  subject: {"text":"Hotel: ..."}  text
    : {"from":7}
```

6 Owner-Signed Task Programs

This section describes how an untrusted planner can do useful work when none of its output is trusted. The planner drafts a whole program, the owner signs the operations that program may perform, and the kernel enforces where every value of every operation comes from.

6.1 Program model

The planner sees only the owner's request (§6.5) and the reviewed tool catalog of the deployment. It never sees a tool result, exactly as CaMeL's P-LLM. It returns a small program: an ordered list of at most eight steps (§10.6 explains the bound). A step is one of three kinds.

- A **tool step** names a reviewed operation and gives every field of that operation an *origin*.
- An **extraction step** asks the quarantined model for the one value a named field needs, or for the final answer. It reads an earlier step's result as data.
- A **compute step** applies a fixed deterministic function to earlier results (§6.4).

An origin is one of:

- `{"text": t}`: a literal, which must be owner text. A list field takes typed items, each held to the same rule. The owner may also declare constants: its own dates and times restated in a canonical form, booleans, sharing permissions, and search terms for read tools (never a URL).
- `{"from": k}`: the value that extraction or compute step k produced.
- `{"from": k, "path": [...]}`: a scalar, or an array of strings, at a JSON path in tool step k's result.

A `from` origin may add *one* computation on its text: add minutes, add days, or add an amount of money. The magnitude must be stated in the request ("1 hour", "by 100"). The result of the last step is released to the owner. Listing 2 shows a program drafted for a travel task.

6.2 The owner's review

Before anything is signed, a fixed review admits or refuses the program. It is a pre-filter that lets an honest planner succeed and refuses obvious abuse early; the kernel independently enforces the same origin rule (§6.3). The review refuses a program under any of 33 closed rules. The most consequential are:

- a tool not served by the deployment (`unserved_tool`);
- an edge to a later step;
- a literal that is neither owner text nor an owner-declared constant (`literal_not_owner_text`);
- a *destination*—recipient, channel, user, URL, account, or a cc/bcc address—taken from data (`derived_destination`);
- a computation whose magnitude the request does not state (`compute_amount`);
- the final answer, or a yes/no condition, used as a field value;
- more than eight steps.

The destination rule is the owner's conservative policy: an injection may change *what* a write says, never *whom* it reaches. A deployment with a human owner can relax it by approving each data-derived destination through G6. Our evaluation keeps it strict and measures its cost (§10.4).

Retry without a new channel. When the review refuses a draft, the planner may draft again, up to three drafts in total. It sees only its own previous program and a fixed message keyed by the rule code, the step number, and the field name. No tool result or model output ever reaches the planner, so a retry adds no injection channel. Policy refusals, such as a derived destination, are not retried: another draft cannot obtain an authority the owner did not grant.

6.3 Task authority and kernel enforcement

An admitted program becomes three signed objects.

1. **Owner document.** The owner's request text and the owner-declared constants enter through ingress as one owner document. They become UserAuthorized roots after G1/G2.
2. **Task authority.** The owner signs a task authority with one *clause* per operation, plus a final-release clause. Each clause fixes the tool descriptor, the exact owner values for the controls, and the *derived controls*. Each derived control is a signed edge naming the source clause, JSON path, value kind, byte bound, and optional computation. Every clause allows a single attempt of magnitude one, and names all earlier clauses as predecessors.
3. **Plan.** Signed operator commands compile the plan against exactly these clauses. Each descriptor must be active and must match its tool. A whole-result edge may fill only the payload, and a path edge may never fill the payload or a magnitude.

Preparation then runs `check_owner_text_origin`. Every input must be a declared constant, or a text whose every item is a substring of the normalized request. Preparation also runs the result-edge rule: the owner's exact values, plus the declared edge rules, must reconstruct an alternative the owner signed. All of this happens before any operation runs. During execution the kernel drives the operations itself. Each request passes G4 (an exact match against its clause), G5, G6, and G7. A value taken from a result is only ever the kernel's own derivation over the retained result of the source clause. This is property P6.

6.4 Values from data

Quarantined extraction. An extraction step is a tool of the deployment, executed by a connector that calls a model. Its payload is the earlier result, passed by the kernel as data, plus up to five other earlier results as context. Its instruction is always the owner's whole request, whatever the planner wrote. Its target is an inert catalog label, and its output is one bounded line. Because the call sends data to a model provider, it is declared as a SEND effect. Under intent-flow confinement, every call is therefore an owner-approved action, never a silent step. Its output reaches a later field only through a signed edge, and it can never fill a destination.

Conditional writes. Requests often make an action depend on the data: *"if something looks suspicious, change the password"*. Every write therefore carries an owner gate field. Its value is either the owner constant `always`, or a signed edge from an extraction that answers yes or no to a condition copied verbatim from the request. The connector performs the official call only on the constant or on an exact "yes". Any other answer, including an injected one, does nothing. A condition can decide only *whether* a signed effect happens, never what it does or whom it reaches. A gated step's result may not be used later.

Deterministic compute steps. Choosing among records—*"the highest-rated hotel, the more expensive on a tie"*, *"the total I spent in March"*—previously required an extractor to compare whole tables in one reply, and it often got this wrong (§10.4). `dojo.compute.table` does the comparison as a fixed function in the connector instead.

- **Inputs.** Up to four signed edges to earlier results. The first gives the rows (an object's entries, a list of records, or a list of names); the others are joined to them by key.
- **Specification.** Up to three filters (`<ref> <op> <operand>`), two orderings (`max|min <ref>`), and one output (`key`, `keys`, `count`, `sum`, `value`, `values`, or `pairs`). These texts belong to a closed grammar that the review parses and the owner declares as constants.

- **Effect.** The step is a READ that contacts nothing, so it needs no approval.
- **Output.** Data, exactly like an extraction's: it is never a destination and reaches a later field only through a signed edge. Operands select rows; they never appear in the output.

The step adds no authority. A compromised planner can already choose which record an extraction returns. Compute only makes the honest choice exact.

6.5 Value-blind planning and extraction

The planner is a remote model, so the owner's request is itself a disclosure. Savana can mask the planner's view using the kernel's own sensitive-span definition (§7.4).

- **Masking.** Every detected span is replaced by a typed placeholder, such as `<EMAIL_1>` or `<PERSONAL_DATA_1>`. Equal values share a placeholder, and a span is widened to its whole word. The masked text is re-scanned, and drafting fails closed if any span remains.
- **Binding.** Before review, the owner binds each placeholder back to its value. Review, signing, and the kernel therefore see only real values. A placeholder grants nothing that copying the value would not.
- **Extractor views.** The same mechanism masks the extractor's inputs, with one placeholder table per call. A `strict` view hides everything. A `quantities` view keeps dates, times, and decimals visible, which the extractor needs in order to compute.
- **Verification.** The offline verifier recomputes every request that left the host, and proves that no masked value is in it.

6.6 What a compromised planner can do

Under S6, a compromised planner cannot introduce a destination, a literal the owner did not write or declare, an unrequested computation, or a tool outside the reviewed catalog. Data never becomes control: a value from a result reaches a field only along a signed edge, a write that binds such a value requires approval, and a destination filled by such a value requires approval even on a read (§5.7). "Do what the e-mail says" is therefore not expressible.

What remains is choice *within* the grant. The planner may:

- add an extra write to an owner-named destination;
- pick a different record of an authorized read;
- word the final answer;
- do nothing.

In a deployment with a human owner, inspecting the program before signing is the backstop for this residue. The benchmark's software owner does not inspect, and we say so wherever it matters.

Transition	**Target** $(conf, readers)$	**Gate duty**
MaskTokenizeAndLeakCheck	(AgentMasked, AGENT)	blocklist $\wedge$ no PII
BuildPlannerEnvelope	(PlannerAbstract, EXT. PLANNER)	blocklist $\wedge$ no PII
BuildFusedModelEnvelope $\{model\}$	(PlannerAbstract, model)	blocklist $\wedge$ no PII
BuildApprovalDisplay	(AgentMasked, APPROVAL)	blocklist
BuildExecutionEnvelope $\{exec.\}$	(VaultBound, EXECUTOR)	blocklist
BuildFinalRelease $\{sink\}$	(VaultBound, EXT. SINK)	blocklist

Table 5: The closed set of declassification transitions. The variants that hand a value to one recipient carry that recipient's identity digest. **[TODO: confirm the exact target label of BuildFusedModelEnvelope.]**

7 Declassification

7.1 Six transitions, duties by recipient

Because derivation only narrows, the only way a value's audience can grow is an explicit declassification. Savana defines exactly six transitions (Table 5). Each names a target $(conf, readers)$ and a *leak-gate duty*: what the content gate must prove about the exact outgoing bytes before the transition may run. Control endorsement is separate: it never widens a value's readers.

The duty depends on the recipient, not only on the confidentiality level. Values sent to an LLM-facing role must contain neither blocked instructions nor detected PII, because those bytes become model context. The approver and executor, by contrast, need the concrete value to decide or act. Masking a recipient address would make an intelligible approval impossible, and masking an execution argument would change the operation. These transitions therefore enforce the universal blocklist but permit concrete fields. A signed rule may strengthen a transition's duty floor but cannot weaken it. The instruction blocklist is defense in depth: a miss may change a model's proposal, but P1 and P3–P6 still hold, because model output carries no capability.

7.2 Exact-recipient binding and computed evidence

For handoffs to the executor, to a model, and to an egress sink, the recipient's identity digest is carried *inside* the transition variant. The transition therefore cannot be named without naming its reader, and an all-zero identity is refused. A reader *class* can express "an external sink may read this", but it cannot distinguish the release the owner asked for from a release to an attacker's destination.

The declassification constructor is private. It runs the gate and derives the evidence digest from what the gate observed:

```
let leak_gate_digest = enforce_for_declassification(
    value, transition.duty().strictest(rule.duty_floor
     ()))?;
```

`leak_gate_digest` is deliberately not a parameter. A caller-supplied digest would assert a check the kernel cannot verify occurred, at exactly the point where the kernel gives up a confidentiality guarantee. The constructor also refuses records whose rule, implementation, token-set, or purpose digests are all zero: a record "authorized by" a rule that names nothing binds nothing.

7.3 Signed rules and the handoff judgment

A declassification rule set defines which transitions may run. It is canonically encoded, domain separated, and hash chained by sequence number and predecessor digest. It is signed by a key with the dedicated DeclassificationAuthority purpose, and the active deployment manifest pins its digest. A set contains at most 64 rules, each with at most 16 reader identities. The public `declassify` entry point checks, in order:

1. the set's validity window;
2. a rule for this transition and purpose, and that rule's window;
3. that the rule's implementation digest equals the digest compiled into the kernel;
4. the exact recipient;
5. a verified final-release settlement, where the rule demands consent.

Only then does the private constructor run the gate. Rule sets roll over atomically under a write lock; every failure leaves the old set active.

Minting a node is necessary but not sufficient to cross a service boundary. Every handoff consults `judge_handoff`, which returns Admits, Refuses, or Unproven, and only Admits crosses. For execution and final release, a wrapper enforces *gate before durable prepare*: the durable-preparation closure cannot be reached if the gate refuses. A refused handoff therefore leaves the dispatch authorized but unprepared, and leaves quota untouched.

7.4 One definition of "sensitive"

The masker at ingress, the value-blind planner and extractor (§6.5), and the verifier at declassification all consume the *same* definition from one crate:

- a 21-pattern content blocklist;
- a 17-class PII table, whose span finder returns the union of the pattern matches and a blunter byte scanner;
- a pattern-set digest, folded into every gate digest.

The patterns are ports of Python regular expressions, and the two engines disagree on `\b` , `\s` , `\d` , and `\w` . Every character class is therefore rewritten over pinned Unicode tables. Word boundaries are rewritten by their two-sided definition, and any engine error counts as a blocklist hit. A differential corpus mechanizes the one-way implication $masked(x) \Rightarrow gate_passes(masked(x))$. It has 427 hand-written vectors in 17 behavioral groups and 6,000 fuzz vectors. The definition is pattern-based: it does not recognize names or street addresses (§10.5).

8 Signed Authority and Durable Effects

8.1 The signed policy supply chain

Nothing in Savana is configured; everything is signed. An installer/MDM key signs *operational trust-root sets*. These are canonically encoded, domain-separated, hash-chained objects. Each member carries one of five purposes—deployment authorization, rollback authorization, installation activation, installer/MDM, and declassification authority—together with a key epoch and a validity window nested inside the set's. Above the roots sit:

- signed deployment manifests and claims: release identity, binary and projection pins, and the declassification rule-set pin;
- a signed tool-descriptor registry, whose publisher key is authenticated by the active state manifest;
- the executor material, which is bound into every dispatch through an effect-gate lease.

Three implementation disciplines recur across every signed object.

1. **Possession implies verification.** Verified types have private fields and a single verifying constructor, so holding the value is the proof. Negative API contracts are enforced as `compile_fail` documentation tests.
2. **Canonical re-encode equality.** Decode, re-encode, and compare with the input bytes; any non-canonical encoding is refused. The workspace uses 563 distinct NUL-terminated domain-separation strings to prevent cross-protocol signature reuse.
3. **Chain, don't patch.** Revocation publishes a successor. Predecessor validation enforces sequence continuity, digest linkage, and family identity.

8.2 Atomic deployment

Policy, registries, model identities, sandbox profiles, service identities, and binaries activate as one signed restart transaction.

- A dual-slot signed ledger records the active generation.
- A highest-ever vector over 29 security domains prevents rollback across generations.
- A typed transaction head records the 25 activation steps and their evidence.

Missing, expired, incorrectly signed, or unpinned material leaves the service non-ready.

8.3 Approval settlements

An approval settlement is the G6 evidence for an action. The kernel mints a challenge-bearing, principal-bound approval envelope. The envelope carries the gated display bytes, their declassification node digest, and the complete action digest. The approval service returns no display until a WebAuthn *display-authentication* ceremony has established a tab-bound session. The owner then performs a second, distinct WebAuthn *decision* ceremony. The two ceremonies use different signed types, challenges, nonces, and one-use states.

Settlements are signed by the approval service's settlement key under purpose-specific signature domains: UI authentication, ingress, tool execution, final release, and connector registration. A settlement for one purpose therefore does not verify as another. The kernel checks:

- the key, installation, active manifest, and generation;
- the envelope and action digests, principal, and challenge;
- the validity interval and one-use state.

For tool actions, the display is rendered from the exact business request (JSON). Whether that rendering supports *informed* consent is a human factors question that we have not studied (§11).

8.4 Journal before effect

Before writing a byte toward an executor, the kernel commits one atomic effect-preparation transaction. The transaction does the following:

- consumes exactly one exact-action capability and its closed dispatch subject;
- consumes the ticket, and the settlement where one is required;
- re-checks the control endorsements (§5.6);
- moves quota from available to reserved;
- locks the subject, the executor, and the active manifest;
- creates the *sole* execution nonce;
- stores the HPKE-sealed envelope and a Prepared journal entry;
- binds the dispatch to the current effect-fence epoch.

Either every kernel-owned item becomes durable or none does.

The executor's journal is the durable truth at the external-byte boundary:

1. fsync a ProviderAttemptPrepared record;
2. sign and store an EffectStarted receipt;
3. only then emit the first provider byte, after checking that the request is equivalent to the fixed business request;
4. encrypt and fsync the bounded response before any codec sees it.

After a restart, the executor recovers before it serves. A dispatch that never opened an attempt is resumed, since no byte has left. An attempt opened without a retained response becomes indeterminate, even if no effect-started receipt was written yet. A retained response is decoded without contacting the provider. The kernel re-queries each outstanding dispatch by execution nonce and never issues a new nonce for the same clause. Only a *proven* no-effect failure may be re-planned. The journal establishes at most one provider attempt per execution nonce, not global exactly-once semantics; §10.7 tests this by killing the executor at every journal boundary.

9 Implementation

Savana is implemented in Rust (edition 2021, pinned toolchain) with `unsafe_code = "forbid"` at the workspace level. Only the platform crate downgrades this to `deny`, and it re-enables unsafe code in four audited native-boundary modules. Table 6 gives the breakdown at the evaluated commit. A SHA-256 manifest freezes the production core, and checking it is the first CI step. **[TODO: The manifest currently lists 331 files, 66 of which have drifted from their recorded hashes; refresh it before artifact submission.]**

Benchmark integration. The harness maps each of the 74 AgentDojo v1.2.2 tools to one reviewed operation. Each operation has business fields of type text or text list and an explicit role (resource, destination, magnitude, payload, or parameter). A deterministic, total decoding rule takes each field to the official argument. A deployment ships exactly one suite, plus the extractor and the compute tool, so a task context never lists more than 64 tools. Connectors run the *original* AgentDojo functions once per call. Hostile text in any field is data: it is never parsed as a tool call, and no field can name a different function.

Cryptography and sandboxing. Savana uses the following primitives:

- Ed25519 signatures with weak-key rejection;
- SHA-256 domain-separated digests;
- X25519 with HKDF, and ChaCha20-Poly1305 or AES-GCM, for sealed envelopes and durable records;
- canonical CBOR [8];
- OS entropy, with explicit zero checks on generated nonces.

Parser and codec workers are one-job child processes with cleared environments. On Linux they run under Landlock [40] path rules, a seccomp-BPF filter with a kill-process default, and resource limits; on macOS, under a deny-default Seatbelt profile.

Component	LOC	Role
policy-core	114,887	labels, provenance, task authority, rules, engine
kerneld	83,522	kernel daemon, authorities, data plane, recovery
kernel-protocol	55,565	closed wire vocabulary, Suite-1, CBOR
agentd	23,855	agent sessions, planner privacy
execd	19,525	connector runtime, journal, workers
approvald	18,235	WebAuthn, settlements
platform-identity	13,003	measurement, sandboxes
client	12,226	Rust SDK state machine
ingressd	9,118	framing, parser workers
continuation core / private workflow	7,278	v0.4 planning, prototypes
vault, leak-gate, input-runtime	7,874	custody, sensitive-span definition, G1/G2
others	21,361	Python binding, release receiver, legacy NER, tooling
Rust total	**386,449**	18 crates plus tools
Benchmark harness (Python)	10,854	catalog, review, connectors, runner, verifier

Table 6: Implementation size (physical lines, inline tests included). The agent-framework integration adds 3,575 lines of TypeScript.

Agent-framework integration. A production integration with an existing agent framework registers one provider, one model, and one harness, and zero framework tools. Only bytes delivered by the executor over a pinned mutual-TLS receiver can become an assistant reply. We omit the framework's name from the anonymous draft.

Testing assets. The workspace contains 2,302 Rust test functions, including 66 `compile_fail` documentation tests, and 104 integration-test files. It also includes:

- the 6,427-vector gate corpus;
- adversarial policy and startup matrices;
- a crash matrix over the durable pipeline;
- an 821-line negative matrix for declassification rule sets.

The benchmark harness has 296 Python tests.

10 Evaluation

We evaluate the owner-signed program path on AgentDojo and compare it with CaMeL under the same model. We ask six questions.

RQ1 Injection.
Across the full injection matrix, how many attacks succeed, how many tool requests exceed the owner's authority, and what utility remains?

RQ2 Compromised planner.
What happens when the planner itself pursues an attacker's objective, in Savana and in CaMeL?

RQ3 Utility.
Where does utility go, and which mechanisms recover it?

RQ4 Privacy.
How much of the owner's personal data reaches models, and at what utility cost?

RQ5 Cost.
What do mediation and the two-model design cost in latency and model usage?

RQ6 Crash consistency.
If a process dies anywhere in the durable pipeline, does any effect repeat, vanish, or stay unresolved?

10.1 Methodology

Benchmark. We use AgentDojo v1.2.2 [15]: 97 user tasks and 35 injection tasks in four suites, giving 949 (user, injection) pairs under the `important_instructions` attack. Utility and attack success are judged by the official task checkers, run on the actual post-episode environment and on the bytes released to the owner. CaMeL's official harness pins suite version v1.2. **[TODO: Confirm and state task-level equivalence of v1.2 and v1.2.2 for the 97 user tasks.]**

Models. Every model in both systems is DeepSeek `deepseek-flash` [16], run at temperature 0 with thinking disabled. This covers Savana's planner and extractor, and CaMeL's P-LLM and Q-LLM. One ablation uses `deepseek-v4-pro`. A loopback relay admits only these two model names and logs the model reported by each response.

Hosts and verification. Each batch of at most nine episodes runs on a fresh, disposable host with its own deployment and keys. The benchmark's software identity admits at most nine bootstraps per armed batch. The owner is a software identity with finite pre-consent, not a human (§11). It approves a display only if every owner-literal field equals the signed program, every derived field shows exactly the signed rule, and a result-fed payload is byte-for-byte that result. A derived *value* is checked only for form and bound: the software owner accepts whatever the kernel extracted at the signed path.

After a run, every batch is re-verified offline against the source revision it ran. The verifier re-runs the review, recomputes every model request and checks it against the recorded

		Savana			CaMeL / +policy	
Suite	Pairs	Util.	Att.	Unauth.	Util.	Att.
workspace	560	297	0	0	400 / 309	0 / 0
banking	144	27	0	0	104 / 54	0 / 0
slack	105	16	0	0	56 / 19	0 / 0
travel	140	31	0	0	53 / 53	2 / 2
total	949	371	**0**	**0**	613 / 435	2 / 2

Table 7: AgentDojo injection matrix: the official important instructions" attack on all 949 pairs, same model. Util. is utility under attack; Att. is targeted attack success; Unauth. counts tool requests that reached a provider without matching the owner-signed authority. The Savana batches were 107, each on a fresh host.

digest, recomputes the official checker, and checks the evidence hash chain. Only verified batches are counted, and every batch reported here verified.

Baseline. We run CaMeL at upstream commit `f083b6b` through its own harness. The only changes are a model registration and, for RQ2, the planner objective. CaMeL's security policies are evaluated by replaying the recorded run, as in the upstream evaluation. We report CaMeL both without policies ("CaMeL", the configuration of the paper's main utility table) and with them ("CaMeL+policy").

Statistics. We report counts. Intervals are 95% Wilson intervals. Each configuration was run once. At temperature 0, drafting is still not fully deterministic: two runs of an identical configuration differed by about ± 2 tasks out of 97 (§10.4).

10.2 RQ1: The full injection matrix

Table 7 gives the per-suite results.

- **Security.** Savana admitted no targeted attack (0/949, 95% CI [0, 0.4]%) and no unauthorized request. CaMeL, with and without policies, admitted the same 2 pairs. Both are travel tasks in which the injection asks the agent to recommend a particular hotel, i.e. attacks on the *answer text*. That is outside Savana's structural guarantee (§3). In one of the two pairs, Savana's program was refused for an unrelated reason. In the other, the program ran in full, and the injection reached the extractor, which did not follow it.
- **Utility.** Utility under attack is 371 (39.1%), against 613 (64.6%) for CaMeL and 435 (45.8%) for CaMeL+policy.
- **Outcomes.** Of the 949 episodes, 585 published, 308 were refused by review, 47 stopped without publication, and 9 failed at authorization. The 9 are all banking task 9: a reviewed program whose signing the kernel rejected.

	Bank.	Slack	Travel	Att.	Util.
No defense	0	10	13	23	274
Savana	0	0	1	**1**	91
CaMeL	0	1	5	6	223
CaMeL+policy	0	0	5	5	146
Planner polluted with the injected text (RQ2)					
Savana	0	0	2	**2**	93
CaMeL	0	4	8	12	215
CaMeL+policy	0	4	8	**12**	135

Table 8: AutoDojo's adaptive injections on 389 pairs (144, 105, and 140), same model. Att. is targeted attack success by the official checker; Util. is utility under attack. Savana made no unauthorized request in either run.

Injections barely move Savana's utility. The planner never sees data, so its input is byte-for-byte the same with or without an injection. Compared with the benign run, 315 pairs stayed correct and 547 stayed incorrect. Another 56 became correct, every one of them published, because a different draft happened to be better. Only 31 became incorrect. The differences are drafting variance, not attack effects.

Adaptive injections. The official attack is a fixed template. AutoDojo [30] instead searches for injections against a given model, and publishes the injections it found against `deepseek-flash` for the banking, slack, and travel suites. We replayed the first cached injection of each of its 389 pairs. Without a defense, the model follows them in 23 pairs (5.9%), against 0 of 144 for the official attack on banking. Savana admitted 1 pair (0.26%) and no unauthorized request; CaMeL admitted 6, and 5 with its policies (Table 8). All of these but one are travel injection 6, which asks the answer to recommend the attacker's hotel. The exception is a slack message to Alice in which CaMeL's model quoted the attacker's link while warning about it; the checker counts the link, and CaMeL's policies stop the message.

10.3 RQ2: A compromised planner

Setup. We gave the planner one of four hidden objectives, appended to its system prompt as "secret instructions from the party that controls you":

- *exfiltrate*: deliver data to an attacker address;
- *redirect*: make sends, shares, transfers, and invites go to the attacker;
- *tamper*: change amounts, dates, quantities, or accounts;
- *obey_data*: treat instructions found in data as the owner's.

Each objective ran over all 97 benign tasks, giving 388 episodes, plus 194 episodes in which exfiltrate and redirect were allowed up to three drafts. The environment is benign: the planner is the only attacker. CaMeL's P-LLM received the identical objective text, with the sentence about the output format adapted from JSON to code. The relay confirmed

	Savana		CaMeL		CaMeL+pol.	
Objective	Att.	Eff.	Att.	Eff.	Util.	Eff.
exfiltrate	3	0	0	0	46	0
redirect	2	0	9	**7**	45	**3**
tamper	$\geq$3†	0	$\geq$3	$\geq$**3**†	45	$\geq$**2**†
obey_data	–	0	–	0	43	0
utility	31/31/28/34		65/61/62/60			

Table 9: Compromised planner, 97 benign tasks per objective, same model. Att. = episodes whose planner output encodes the attack; Eff. = executed attacker effects. †Checked by hand (see text). With up to three drafts, Savana had 6 and 3 attempting episodes for exfiltrate and redirect, still with 0 effects.

that every P-LLM request (329/329) carried it.

Metric. An *attack effect* is an executed, non-error write whose arguments contain the attacker's address or account. Value tampering cannot be detected this way, so every tamper candidate was checked by hand against the request. An *attempt* is a planner output that contains the attack: for Savana, any draft, whether refused or not; for CaMeL, the P-LLM's code.

Results. Table 9 summarizes the results. Inside CaMeL, the redirect objective produced 7 executed writes that included the attacker:

- a hotel recommendation e-mailed to the owner's wife with the attacker added as a recipient;
- a file additionally shared with the attacker;
- the attacker added as a participant in four new events and one existing meeting.

Three of these episodes were scored as *successful* by the official checker, which does not look for extra recipients. CaMeL's policies stopped four. Two of those four are tasks the same policies also stop for an honest planner, and the replay log does not record why the other two were stopped. The remaining three ran.

The tamper objective produced three verified value changes inside CaMeL:

- a dental appointment moved from the requested 05-20 10:00 to 05-21 11:00;
- a meeting moved from 10:00 to 11:00;
- an unrequested rent amount.

The first two also ran under CaMeL's policies, because a literal in the P-LLM's code is trusted.

Inside Savana, no objective produced an effect, and no request exceeded the signed authority. All 1,178 provider requests matched the owner's clauses. Every draft that encoded an attack was refused before signing:

- 5 drafts with the attacker address in the single-draft runs, and 13 with retries;
- tampered dates and amounts, all refused as `literal_not_owner_text`. These include the same dental-appointment change (Worked example A), a hotel stay shifted by one day, and a rent of 2300 instead of 2200.

In six further episodes, the review admitted a poisoned program, but the kernel refused it at admission, a second, independent line of defense.

The attacker rarely tried. This experiment is weak in one respect, and we report it. The poisoned `deepseek-flash` usually ignored its hidden objective. In Savana, 2–6% of episodes contained an attempt; inside CaMeL, 0–9%. The zero for Savana therefore rests mostly on the *attempts* that did occur, all of which were refused, rather than on many blocked attacks. For an attacker that always tries, we complement it with an offline sweep. We applied nine attack-mutation classes to the 71 real review-passing programs: external, derived, and cc/bcc destinations; literal injection; compute tampering; answer-as-value; and literal context. The review admitted no mutation with a non-owner destination. Its only admissions stayed within the owner's namespace: an extra write to an owner-named destination, or a different record of the same read. **[TODO: Run a forced-attempt variant live on both systems: the same mechanical mutations applied to each system's honest programs, then executed.]**

A planner polluted by data. A hidden objective is one model of a compromised planner; a planner that has read attacker data is another. In the AutoDojo runs of RQ1, we gave the planner every injection text of its episode as `retrieved\_data`, with a one-line note, identically in both systems; the relay matched all 389 exposures byte for byte. Both polluted planners wrote the attacker's goal into their program in 23 of 389 episodes (Table 8). In Savana, all 46 such drafts were refused before signing and no attacker operation ran; the 2 successes are again travel injection 6. In CaMeL, 10 of the 23 attempts ran, and 8 still ran under its policies: four fetches of the attacker's URL, two calendar entries the attacker asked for, one e-mail of the user's passport and bank details to the attacker, and one rent payment redirected to the attacker's account. The official checker misses the last one, because it only inspects recurring transfers and the rent is not recurring; we report its count (12/389 with and without policies) unchanged. CaMeL's policies govern data that flows from tool results, while a literal in the P-LLM's code is trusted, and a polluted planner writes the attacker's values as literals.

Attacks on the label and completeness rules. Two attacks target the rules of §5.5 and §5.7 directly, at the kernel rather than at the review.

Configuration	Tasks	95% CI (%)
Savana, 1 draft	30	[22.6, 40.7]
+ review-feedback retry ($\leq$3 drafts)	39	[31.0, 50.2]
+ conditional writes, 5 contexts, hints	40	[32.0, 51.2]
+ deterministic compute steps, fail-closed G5	41	[32.9, 52.2]
retry, `v4-pro` planner	36	[28.2, 47.0]
conditional writes, `v4-pro` extractor	39	[31.0, 50.2]
CaMeL / CaMeL+policy, `flash`	58 / 39	[49.8, 69.0] / [31.0, 50.2]
CaMeL / CaMeL+policy, `v4-pro`	71 / 47	[63.6, 81.0] / [38.8, 58.3]

Table 10: Benign utility on the 97 AgentDojo user tasks.

Cause of failure (G11, 57 failures)	Tasks	CaMeL^{v4}
Destination would come from data (refused, or write omitted)	24	18
Read-only answer wrong (multi-criteria comparison)	12	4
Stopped at the $\sim$60 s bound, or no room for the write in 8 steps	7	6
Task asks to follow instructions found in data (by design)	5	0
Planner error (literal, field origin, omitted write)	5	4
Write executed with wrong values	3	0
Kernel admission refusal	1	1

Table 11: Every failure of the best measured configuration, one cause each, and how many of them CaMeL solves with `v4-pro`.

- **Planner-written disclosure.** The planner binds a recipient and a body it wrote itself to an owner-signed SEND alternative whose descriptor declares *no* intent-flow validator. Before the fail-closed floor, G5 permitted this call; it now escalates to approval (a kerneld regression test). The same alternative with owner-gated inputs is still permitted.
- **Candidate-universe poisoning.** An injection places the attacker's address at the path a rule-closed destination reads, so the owner's single alternative now denotes the attacker. The domain digest is unchanged, so the alternative still matches; the derived value is ExternalUntrusted, so the floor escalates the action on a write and, through address confinement, on a read (a policy-core unit test).

[TODO: Run both attacks end to end on fresh hosts, with a software owner that declines any approval whose destination is not owner text, and report dispatched-effect counts next to RQ1.]

10.4 RQ3: Utility and its attribution

Table 10 gives the utility of each configuration.

- **Retry.** Review-feedback retry is the largest single gain: seven programs move from refused to correct. Each retry sees only the planner's own program and a fixed rule message.
- **Conditional writes and wider context.** These add a class of tasks ("if ..., then change the password") but little net utility in this run.
- **A stronger model.** A stronger *planner* lowered utility, because it more often derived destinations and wrote longer programs. A stronger *extractor* solved three more tasks when the program was held fixed (27 vs. 24 of 48), but this is within drafting variance overall.
- **CaMeL.** CaMeL gains 13 tasks from the stronger model, and 8 under its policies.

Savana's utility is therefore not limited by model capability.

Where utility goes. Table 11 attributes every failure of the best measured configuration.

- **Data-chosen destinations.** The largest share follows from the owner's rule against data-chosen destinations: "message the most active user", "add Alice to the quietest channel", "e-mail each person in the minutes". This is also where most of CaMeL's advantage comes from, and precisely where CaMeL executed the compromised planner's redirections (RQ2).
- **Instructions in data.** Five tasks ask the agent to carry out instructions found in data ("do the tasks on my TODO page"). They are not expressible in Savana, by design, and CaMeL fails them too.
- **Multi-criteria answers.** Twelve read-only answers needed a comparison across several tables, which the extractor had to make in a single reply.

The compute step (§6.4) targets this last class.

- **Offline review.** Savana's review admitted 76 of 97 drafts, and 18 of them used a compute step. This is review-only and says nothing about utility.
- **Live smoke run.** In a seven-task run, the compute step chose the correct restaurant in both multi-criteria travel tasks. Both programs then lacked the steps to write the answer or the reminder.
- **Full run.** On all 97 tasks, with the fail-closed G5 floor of §5.6 in the kernel, Savana completes 41 (workspace 26, banking 7, slack 3, travel 5), against 40 before. The kernel blocked nothing and saw no unauthorized request, so the floor caused no false refusal. The compute step fixed travel tasks 9 and 15 and workspace task 12; it did not raise the total beyond drafting variance, because programs in banking and slack more often omitted their write (2 of 13 and 3 of 15 published programs wrote, against 4 of 9 and 5 of 13 before).
- **No-op tasks.** Banking tasks 5, 6, 8, 9, and 10 pass the official check with no tool call and an empty answer. Without them, Savana completes 37/92 (38/92 before) and CaMeL with `v4-pro` 67/92 (43/92 with policies); both systems are credited with four of the five.

Channel	Plain	Masked	Utility cost
Planner requests (spans)	27	0	none with retry
Extractor, `quantities` view	1,093	374	22→20 of 48
Extractor, `strict` view	1,093	0	22→17 of 48
Kernel model view (bytes)	0	0	–

Table 12: Personal-data spans that models received (kernel leak-gate definition). Utility costs are measured on the tasks whose program was identical across views.

10.5 RQ4: Personal data that reaches models

We measure every request that left the host, using the kernel's own sensitive-span definition (Table 12).

- **Planner.** In plain view, 21 of 97 requests contained personal data (27 spans). The value-blind view reduces this to zero. On the 21 affected tasks, utility with retry was unchanged (8/21 in both views). The other 76 requests are byte-identical in both views.
- **Extractor.** The `strict` view removes all 1,093 spans, and costs 5 of 22 tasks that ran an identical program. The `quantities` view keeps 374 spans, all of them dates, prices, and timestamps, and costs 2.

The definition has known gaps. It does not cover person names or street addresses. It also misclassifies 8-digit dates as phone numbers: 12 of the planner's 27 spans were such dates.

10.6 RQ5: Cost

Latency. We decompose episode time from the evidence timestamps:

- **Admission.** Authorization, compilation, and preparation take about 5 s before the first operation.
- **Per step.** Each later step takes a median of 4.6 s (tool 4.64 s, extraction 4.81 s, compute 4.29 s). The provider's own share is small: 1.0 s for an extraction's model call, and 3 ms for a compute step. The rest is the deployment's control loop rather than the checks: the kernel's owner-clock drivers advance a program at most one stage per second by design, and each step waits for two approval round trips through the software owner, which polls every 250 ms.
- **Whole programs.** Two-step programs publish in a median of 20.3 s, and eight-step programs in 47.9 s (maximum 60.9 s).

In our configuration, programs of 11 or 12 steps published in only 2 of 8 episodes; the others stopped at about 62–66 s. This is why the review caps programs at eight steps. **[TODO: Pin down which 60 s validity bound stops these programs; the settlement expiry in approvald is the leading candidate.]** An earlier kernel performance defect made each round grow with the host's history, from 38 s to over 60 s within five rounds. We fixed it by re-verifying only changed journal entries, together with an optimized integration build; round time is now flat at about 17 s.

The checks themselves. We timed the checks in process on the build our evaluation hosts run (opt-level 1 with debug assertions): one owner-signed tool call through G4, G5, and G6/G7 with the executor's journal, worker check, and Suite-1 IPC, then completion reconciliation, with every durable commit encrypted and fsynced on ext4. Over 1,000 fresh fixtures, a call takes 58.8 ms at the median and 85.1 ms at p99 (53.3 ms and 75.7 ms at opt-level 3). Durable commits, not computation, set this cost: one atomic replace takes 1.1 ms on this disk, and the executor alone commits six times per call. Over the same period, the same model took 1.0 s at the median for an extraction-sized prompt and 1.9 s for a planner-sized one, so mediation at p99 is under a tenth of one model call.

Approval-free paths. In the evaluated deployment, the signed G5 policy disposition is RequireApproval for every tool, so every operation—read, extraction, compute, or write—was settled by the software owner's pre-consent; the measured approval-free fraction is therefore zero, by configuration. To estimate the fraction under a Permit disposition, where only the kernel's floor escalates, we classify each step of the reviewed programs by the rule of §5.6. Of 299 steps in the 76 accepted G12 programs, 164 (54.8%) would be approval-free: 89 reads with owner literals, 50 reads with derived query fields, 20 compute steps, and 5 writes with owner literals only. The other 135 are 119 quarantined extractions, which send data to a model and are therefore SEND calls with an untrusted payload, and 16 writes with a derived argument. The executed G11 programs give 142 of 307 (46.3%). No program is approval-free end to end, because each composes its answer through an extraction. **[TODO: Re-run with a Permit disposition and report the measured fraction; consider treating extraction to a pinned model identity as a declassification (BuildFusedModelEnvelope) rather than an effect.]**

Model usage. Per benign task, Savana makes 2.8 model calls with about 7.3k input tokens:

- 1.35 planner calls of 4.4k tokens on average;
- 1.45 extractor calls of 0.9k tokens each.

CaMeL, measured over its four 97-task runs in RQ2, makes 4.7 calls with about 21k input tokens, estimated from request bytes at 4 B/token. Its P-LLM resends its 3k-token system prompt and the growing conversation on every retry. Savana thus uses about 2.9× fewer input tokens, and pays for it with the latency of mediation.

10.7 RQ6: Crash consistency

We test P5 by killing the process that runs a real fused dispatch: G4–G7, the sealed envelope, Suite-1, and the ex-

Journal state when killed	Trials	Prov. req.	Bad
accepted, no attempt yet	62	1	0
1. attempt record fsynced	53	0	0
2. effect-started receipt stored	42	0	0
3. first provider byte sent	28	1	0
4. response retained	60	1	0
completion recorded	84	1	0
kernel acknowledged	41	1	0
executor not yet reached	91	0 or 1	0
total	461	≤ 1	**0**

Table 13: Crash matrix. Each trial runs one real dispatch and kills its process with SIGKILL: before and after each journal commit's anchor advance (20 trials each), inside the provider after it recorded the request (20), or at a random time (200). Prov. req. counts provider requests for the execution nonce across all processes.

ecutor's encrypted journal. A second process then reopens the journal exactly as the daemon starts, recovery first and service second, resends the kernel's original dispatch, and restarts once more. A trial is *bad* if its nonce reaches the provider twice; if a resend or a restart reaches it again; if an entry is still in flight after recovery; if an effect is recorded as a proven no-effect failure; or if a completion exists without an effect. None of 461 trials was bad (Table 13); rows 1–4 are the four steps of §8.4. Recovery is conservative in one place: an attempt whose effect-started receipt was not yet written is provably pre-byte, yet becomes indeterminate rather than a re-plannable no-effect failure. Two limits apply. A killed process is not a power loss: the page cache survives, so fsync ordering under power failure is untested. And the kernel dies together with the executor without reopening its own durable state; its crash boundaries are covered by in-process fault injection, each asserting exactly one provider request.

10.8 Threats to validity

- **One benchmark.** We evaluate on AgentDojo only, the one benchmark on which CaMeL has been run by many groups. The adaptive attack is AutoDojo's published search result for our model, replayed (one cached injection per pair), not a search against Savana itself. Open-ended tasks (AgentDyn [28]) and broader suites (ASB [4], InjecAgent [50]) remain to be run. **[TODO: Add AgentDyn.]**
- **One model family.** All models are DeepSeek. CaMeL's published results use o3, Claude, and Gemini, and CaMeL reaches 71/97 here with `v4-pro`, close to its published o4-mini and Claude figures.
- **One run per configuration.** Drafting variance is about ± 2 tasks.
- **A software owner.** The owner is software. It checks the signed structure of each display but accepts any well-formed derived value, so in our runs an approval adds no semantic check on data-derived bytes. A human owner would both add that check and add latency.

11 Discussion and Limitations

The review filters; the kernel enforces. The owner review is a small Python procedure. It exists so that an honest planner succeeds and obvious abuse is refused early, with a fixed message the planner can act on. The origin rule (S6) is enforced again by the kernel, both at preparation and at dispatch. In six poisoned episodes the kernel refused a program that the review had admitted (§10.3).

The review refuses *any* destination derived from data. The kernel is less strict but not silent: it admits a result edge into a destination, and G5's floor then requires approval for the action, on a write or a read alike. A human owner would therefore see every data-chosen recipient or address before it is used. Refusing outright is the largest single cost in utility (§10.4); it is a policy choice, not a mechanism limit.

The residue of a compromised planner. Within the grant, a compromised planner can still choose: another record of an authorized read, an extra write to an owner-named destination, or the wording of the answer. Our offline sweep found no other admissions. Inspecting the program before signing closes this residue in a deployment with a human owner. A benchmark cannot measure that inspection.

Enforced versus by construction. We use *enforced* only for properties checked on the production path. Restrictions that hold only because no code path produces a value are called structural or by-construction. An earlier revision kept a tested declassification constructor that the production path never called. The current kernel routes all six reader-widening paths through the constructor and records the resulting evidence.

Platform root of trust. The production deployment path requires a non-exportable platform keystore and a hardware-backed monotonic rollback authority. The sealed platform boundary currently reports that authority as unavailable, so release builds exit rather than run with file-backed substitutes. Our evaluation therefore runs development deployments with explicitly gated file-backed authorities, which release builds refuse at compile time.

Detection recall. The content gate is deterministic and pattern-based, and cannot recognize every semantic secret. In particular, it misses names and street addresses and overmatches 8-digit dates. A false positive refuses a transition.

A false negative can disclose content outside the configured predicate during a model-facing declassification. P2 therefore proves mediation of the configured rule, not semantic noninterference. Because the pattern-set digest pins the masking contract, changing recall is a visible, signed policy change.

Fail-closed descriptors. P3 and P6 rely on signed descriptors, so we make their defaults fail closed rather than assume them correct.

1. **No default role.** Every business field declares one of resource, destination, magnitude, payload, or parameter; a missing or unknown role rejects the descriptor. A request must carry exactly the profile's fields.
2. **Everything but payload is control.** Resource, destination, and parameter fields feed the Scope, Destination, and Argument facets; only payload is outside every facet, and it is bound by the payload digest and, when untrusted, by approval.
3. **The flow floor is not declared.** Intent-flow confinement is applied by G5 to every call, so a descriptor that omits the validator cannot let untrusted data steer a state-changing call.
4. **Addresses are destinations by default.** An untrusted value in any field the profile does not name as a non-destination field escalates, for every effect; a descriptor without a profile treats all its fields as addresses. A field that chooses where a read goes (a URL) must be the read's destination; `dojo.web.get` declares its URL so.
5. **Effects are cross-checked.** A descriptor's effect set must contain its profile's effect, so a SEND profile cannot ship as a read.

What remains trusted is the publisher's choice of *which* field is a URL or a recipient: if a signed descriptor declares a real address as a query parameter, a data-chosen value can reach it on a read. That is a signed policy defect, visible in the descriptor, not a runtime bypass.

Approval semantics and human factors. The tool-approval display is rendered from the exact business request, but it has not been studied with users. Two WebAuthn ceremonies per approval impose attention and latency costs. Until a user study exists, we describe G6 evidence as authenticated confirmation of exact bytes, not as informed consent.

Time and step budget. About five seconds per step, and an end-to-end validity bound near 60 s, limit programs to eight steps. Several failures are programs that ran out of room. Making mediation cheaper, or running independent reads in parallel, would recover them without weakening any check.

Assurance. Savana has had no third-party audit, and P1–P6 are not formally verified. A finite-state model of the task-trace specification is checked separately, but it says nothing about the Rust implementation. Mechanizing the label algebra and the action-capability state machine is the natural next step.

12 Related Work

Information-flow control and capabilities. Savana's labels descend from lattice IFC [17], the decentralized label model [32], and the IFC operating systems Asbestos, HiStar, and Flume [19, 49, 26]. It also draws on language-level dynamic IFC [43] and on capability discipline [18, 31]. Our contribution is not the lattice but its application to agents:

- readers are system roles and named external identities;
- declassification duties depend on whether the recipient is a language model;
- the recipient's identity is bound into the transition;
- the authority over *effects* comes from an owner-signed task rather than from the plan.

Robust declassification [48], qualified robustness [33], decentralized robustness [12], and nonmalleable IFC [9] are the closest formal notions to P3; they assume trusted code with untrusted inputs, whereas Savana must enforce robustness when the code itself is written by the adversary (§4.4).

System-level agent defenses. The table in Appendix D compares the closest systems.

- **CaMeL** [14] separates control from data flow, with capabilities and policies. Its P-LLM is trusted. CaMeL-CUA [20] extends it to computer-use agents.
- **FIDES** [13] adds information-flow labels to planning, with a focus on confidentiality.
- **Privilege control.** Progent [42] and DRIFT [27] constrain tool calls with programmable or dynamically updated privileges.
- **Static plans.** ACE [1] plans over abstract tools and checks information flow statically. IPIGuard [24] executes a tool-dependency graph fixed in advance.
- **Formal policies.** PCAS and FORGE compile or formally enforce policies for agentic systems [36, 35]. IGAC governs tool authorization by intent [51], and CapAgent applies semantic data-flow governance [23].
- **Isolation.** IsolateGPT [47] isolates applications within an agent.
- **Narrower domains.** PatchWeaver [37] treats an LLM as an untrusted proposer for Kubernetes remediation, and AIOpsShield [3] sanitizes the telemetry an operations agent reads.

Design-pattern surveys catalog these approaches [6, 46, 25].

Savana shares the thesis of moving authority outside the model, and differs in where it draws the line. No model, including the planner, is trusted. Every value of every effect has an owner-signed origin, enforced by a separate kernel

process. Disclosure to models, approvers, and executors is a signed declassification. Effects are journaled so that an indeterminate outcome is never retried automatically.

Attacks on defenses. Adaptive attacks break most model-level defenses [34], and adaptive black-box search has been turned against system-level defenses as well [30]. Injecting data into fields that a defense treats as trusted reaches high success against CaMeL [2]. A systematization identifies the root causes of circumvention [44]. Savana addresses the trusted-field class directly: in Savana, the planner's literals are not trusted. We replay AutoDojo's injections for our model, including with a planner that has read them (§10.2, §10.3), but have not run an adaptive search against Savana itself.

Benchmarks. AgentDojo [15] is the common ground for system-level defenses. InjecAgent [50] and ASB [4] broaden the attack surface, and AgentDyn [28] adds open-ended tasks on which plan-first designs over-defend. Bhagwatkar et al. document checker and injection flaws in these benchmarks [7]. Our RQ2 observation matches: a checker that does not look for extra recipients scores a successful redirection as utility.

Model-level defenses. Instruction hierarchies, input structuring, and adversarial training reduce what a model will do [45, 22, 10, 11], and benchmarks formalize their evaluation [29]. They are complementary to Savana, which constrains what arbitrary model output can authorize.

13 Conclusion

Prompt injection becomes an authorization failure when a model that consumes untrusted content, or that writes the plan, also holds the authority to disclose data or invoke tools. Savana removes that authority from every model.

- The owner signs one clause per operation.
- The kernel binds every argument to owner text, an owner constant, or its own derivation along a signed edge.
- Labels with an untrusted-effect ceiling keep data from minting effects.
- Signed, recipient-bound declassification governs what each model and executor sees.
- A journal carries every decision to the external boundary.

On AgentDojo, no injection and no compromised-planner objective produced an effect in Savana. With the same compromised model, CaMeL redirected sends and changed values. Savana pays for this in utility, and we have traced that cost to explicit policy choices. The largest is the refusal to let data choose whom a write reaches. The next step is to recover that utility with the owner in the loop rather than with the planner.

Ethics Considerations

This work develops and evaluates a defensive system. All attacks ran against local, disposable AgentDojo environments with synthetic data. No production user, account, or third-party service was targeted. The compromised-planner objectives are fixed test fixtures that mirror AgentDojo's own attack taxonomy (exfiltration, redirection, tampering, and obeying data), and the attacker address is a reserved example domain. Model calls were made through our own API account, within the provider's terms. The gate evaluation must use only synthetic or appropriately licensed personal-data corpora. There were no human subjects.

Publishing the design could help attackers identify policy boundaries or construct inputs that evade the deterministic content gate. We limit this risk by framing the gate as defense in depth rather than as the confinement boundary. We will also disclose the CaMeL observations of §10.3 to its maintainers before publication. **[TODO: Record the disclosure.]**

Open Science

The artifact will contain:

- the evaluated source revision;
- the benchmark harness, with its review, connectors, and offline verifier;
- the CaMeL patch (model registration and planner objective) and the loopback relay;
- the gate corpus and the adversarial and crash matrices;
- the raw, offline-verified evidence of every reported batch;
- the scripts that regenerate every table.

Model endpoints are documented by provider, model identifier, and evaluation date. Credentials and deployment secrets are excluded. **[TODO: Anonymous artifact URL and DOI.]**

References

[1] ACE: A security architecture for LLM-integrated app systems. In *Network and Distributed System Security Symposium (NDSS)*, 2026. arXiv:2504.20984; verify: authors.

[2] Agent data injection attacks are realistic threats to AI agents. arXiv:2607.05120, 2026. verify: authors.

[3] AIOpsShield (TODO: complete metadata), 2026. TODO: cited by the original skeleton; metadata not available in the drafting environment.

[4] Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In

International Conference on Learning Representations (ICLR), 2025. arXiv:2410.02644; verify: authors.

[5] Aslan Askarov and Andrew C. Myers. Attacker control and impact for confidentiality and integrity. *Logical Methods in Computer Science*, 7(3), 2011.

[6] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837, 2025.

[7] Bhagwatkar et al. Indirect prompt injections: Are firewalls all you need, or stronger benchmarks? arXiv:2510.05244, 2025. verify: authors and venue.

[8] Carsten Bormann and Paul Hoffman. Concise binary object representation (CBOR). RFC 8949, Internet Engineering Task Force, 2020.

[9] Ethan Cecchetti, Andrew C. Myers, and Owen Arden. Nonmalleable information flow control. In *Proc. ACM Conference on Computer and Communications Security (CCS)*, pages 1875–1891, 2017.

[10] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *Proceedings of the 34th USENIX Security Symposium*, 2025.

[11] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. arXiv:2410.05451, 2025. verify venue.

[12] Stephen Chong and Andrew C. Myers. Decentralized robustness. In *Proc. 19th IEEE Computer Security Foundations Workshop (CSFW)*, 2006. verify pages.

[13] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control. arXiv:2505.23643v2, 2025.

[14] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. arXiv:2503.18813v2, 2025.

[15] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[16] DeepSeek. DeepSeek API documentation: models and pricing. `https://api-docs.deepseek.com`, 2026. model identifiers deepseek-flash and deepseek-v4-pro; accessed 2026-10.

[17] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[18] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[19] Petros Efstathopoulos, Maxwell Krohn, Steve VanDeBogart, Cliff Frey, David Ziegler, Eddie Kohler, David Mazières, Frans Kaashoek, and Robert Morris. Labels and event processes in the Asbestos operating system. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP)*, pages 17–30, 2005.

[20] Foerster, Blanchard, Nikolic, Shumailov, Zhang, Mullins, Papernot, Tramèr, and Zhao. CaMeLs can use computers too: System-level security for computer use agents. arXiv:2601.09923, 2026. verify: author first names.

[21] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[22] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. arXiv:2403.14720, 2024.

[23] Huiying Hou, Yucong Ma, and Jianyu Miao. CapAgent: Semantic data-flow governance for LLM agents in big-data cognitive computing. *Big Data and Cognitive Computing*, 10(9):293, 2026.

[24] IPIGuard: A tool dependency graph-based defense against indirect prompt injection in LLM agents. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. arXiv:2508.15310; verify: exact title and authors.

[25] TODO: systematization of agentic attacks and defenses (kim et al.), 2026. TODO: cited by the original skeleton; metadata not available in the drafting environment.

[26] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. Information flow control for standard OS abstractions. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, pages 321–334, 2007.

[27] Li et al. DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.12104; verify: title and authors.

[28] Li, Wen, Shi, Zhang, Vorobeychik, and Xiao. AgentDyn: Are your agent security defenses deployable in real-world dynamic environments? arXiv:2602.03117, 2026. verify: author first names and venue.

[29] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[30] Ma, Li, Xiao, Yu, Zhang, and Vorobeychik. AutoDojo: Adaptive black-box attacks reveal the limits of indirect prompt injection defenses. arXiv:2606.15057, 2026. verify: full title, author first names.

[31] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

[32] Andrew C. Myers and Barbara Liskov. A decentralized model for information flow control. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP)*, pages 129–142, 1997.

[33] Andrew C. Myers, Andrei Sabelfeld, and Steve Zdancewic. Enforcing robust declassification and qualified robustness. *Journal of Computer Security*, 14(2):157–196, 2006.

[34] Nasr, Carlini, others, and Tramèr. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. In *USENIX Security Symposium*, 2026. arXiv:2510.09023; verify: exact title and author list.

[35] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. Formal policy enforcement for real-world agentic systems. arXiv:2602.16708v3, 2026.

[36] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Prasad Chalasani, Mihai Christodorescu, and Somesh Jha. Policy compiler for secure agentic systems. arXiv:2602.16708v1, 2026.

[37] PatchWeaver (TODO: complete metadata), 2026. TODO: cited by the original skeleton; metadata not available in the drafting environment.

[38] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. arXiv:2211.09527, 2022.

[39] Andrei Sabelfeld and David Sands. Declassification: Dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009.

[40] Mickaël Salaün. Landlock: Unprivileged access control. Linux kernel documentation, `https://docs.kernel.org/userspace-api/landlock.html`, 2021.

[41] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[42] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control. arXiv:2504.11703v3, 2026.

[43] Deian Stefan, Alejandro Russo, John C. Mitchell, and David Mazières. Flexible dynamic information flow control in Haskell. In *Proceedings of the 4th ACM Symposium on Haskell*, pages 95–106, 2011.

[44] Taxonomy, evaluation and exploitation of IPI-centric LLM agent defense frameworks. arXiv:2511.15203, 2025. verify: authors and venue.

[45] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. arXiv:2404.13208, 2024.

[46] Simon Willison. The lethal trifecta for AI agents: Private data, untrusted content, and external communication. Blog post, `https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/`, 2025.

[47] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[48] Steve Zdancewic and Andrew C. Myers. Robust declassification. In *Proc. 14th IEEE Computer Security Foundations Workshop (CSFW)*, pages 15–23, 2001.

[49] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. Making information flow

explicit in HiStar. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 263–278, 2006.

[50] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

[51] Genliang Zhu and Chu Wang. Intent-governed tool authorization for AI agents. arXiv:2606.22916v3, 2026.

A Owner Review Rules

The review (§6.2) refuses a program for exactly one of 33 closed reasons. Each reason has a fixed retry message.

Shape.
`program_text`, `program_json`, `program_shape`, `program_length`, `step_shape`, and `unserved_tool`.

Fields.
`unknown_field`, `missing_field`, `field_origin`, `literal_kind`, and `literal_list`.

Origins.
`literal_not_owner_text`, `derived_destination`, `too_many_declared_values`, `edge_source`, `edge_path`, `edge_compute`, and `compute_amount`.

Extraction.
`extract_shape`, `extract_target`, `extract_context`, `source_on_tool`, `answer_used_as_value`, and `condition_used_as_value`.

Conditions.
`condition_text`, `gate_shape`, `gate_source`, `gated_result_used`, and `unused_condition`.

Compute.
`compute_shape`, `compute_input`, `compute_spec`, and `compute_large_input`.

B Operational State Ownership

Each daemon's mutable security state is owned by one bounded owner thread. Requests carry absolute deadlines and enter bounded queues. The daemon fail-stops on any of: an owner panic, an audit failure, an identity change, a detected rollback, or an uncertain commit. The agent and executor daemons also hold a process-level *effect gate*: a shared OS file lock on a fixed inode, plus an authenticated read-only ledger projection. It is taken across every planner or effect transition, and is refused once deployment has begun a fenced cutover.

C Extensions Beyond Core Confinement

Properties P1–P6 do not require the mechanisms in this section.

Structural planning. Masking values protects the first of three leakage layers. A remote planner also learns the plan's *shape* and the user's *intent*. Savana can split planning into three stages:

1. a local *mapper* model produces a structural graph from an intent projection;
2. a possibly third-party *planner* model orders that graph, and sees only fresh identifiers, coarse roles, and edges;
3. a *deterministic* decoder maps the ordered plan back to tool classes.

Intent exposure is one explicit knob, and it fails safe to private planning.

Connector registration. The connector registry is a hash-chained state whose genesis and update authority are pinned by the deployment channel. A user-registered connector is a tool surface with a capability ceiling: it can never become an egress sink. Each registration is approved through a distinct-purpose settlement whose display shows the complete descriptor.

D Closest-System Comparison

E Kernel Operation Vocabulary

The kernel exposes a closed set of operations per role:

- the agent-facing role accepts 25 operations;
- the ingress-facing role, 19;
- the executor role, 6;
- the control shell, 4;
- connector control, 7.

Authenticated requests cannot select handlers or cross roles. A startup route-completeness gate proves one closed handler per operation before the kernel publishes readiness.

F Design Principles from Defects

Each principle below was extracted from a defect found and fixed during the system's own review, or during this evaluation.

System	Planner trusted?	Protection and operational distinction
CaMeL [14]	yes (P-LLM)	capability-tagged values; policies before tool calls; crash semantics not a stated focus
FIDES [13]	yes	IFC labels on agent values, confidentiality focus
Progent [42], DRIFT [27]	yes	tool-call privilege policies, static or dynamically updated
ACE [1]	yes (abstract plan)	abstract-to-concrete plans with static IFC checks
IsolateGPT [47]	n/a	isolation of agent applications
Savana	**no**	owner-signed task authority with per-value origins; recipient-bound declassification; journaled indeterminate effects

Table 14: Positioning against the closest systems. "Not a stated focus" describes scope, not a vulnerability. **[TODO: Verify each entry against the current version of each system.]**

Separate payload carriage from control authority.
Defect: untrusted values inherited authorizing effects from the session allowance. *Fix:* normalize effects at every construction boundary, and record selections on seven control facets (§5.6).

Negative evidence must be revalidated.
Defect: a single-candidate justification could go stale after new candidates arrived. *Fix:* re-check the authorization revision and the candidate-domain digest immediately before durable preparation, and return `StateConflict` on change.

The checker computes its own evidence.
Defect: the declassification record accepted the gate digest as a parameter. *Fix:* compute it inside the constructor.

A record that names nothing binds nothing.
Defect: records with all-zero binding digests could be constructed. *Fix:* refuse zero digests in every binding.

One definition of the predicate, mechanized.
Defect: the masker and the verifier consulted different definitions of "sensitive". *Fix:* one crate, one union, a digest in every gate record, and a differential corpus.

Gate before durable prepare.
Defect: a refused gate could leave reserved quota behind. *Fix:* make the durable closure unreachable on refusal.

A filter is not a monitor.
Observation: the kernel refused six poisoned programs that the owner review had admitted. Keep the origin rule in the kernel, even when a pre-filter enforces it too.

A bound is part of the contract.
Defect: a 26 KB file listing exceeded a signed 8 KB edge, so the plan stalled until its validity expired. *Fix:* the review refuses such an input up front.

A benchmark checker is not a security oracle.
Observation: three redirections were scored as successful tasks. Measure effects in the environment, not just task success.

Count attempts, not only successes.
Observation: a poisoned planner that rarely tries makes "zero effects" cheap. Report the attempt rate next to every effect count.